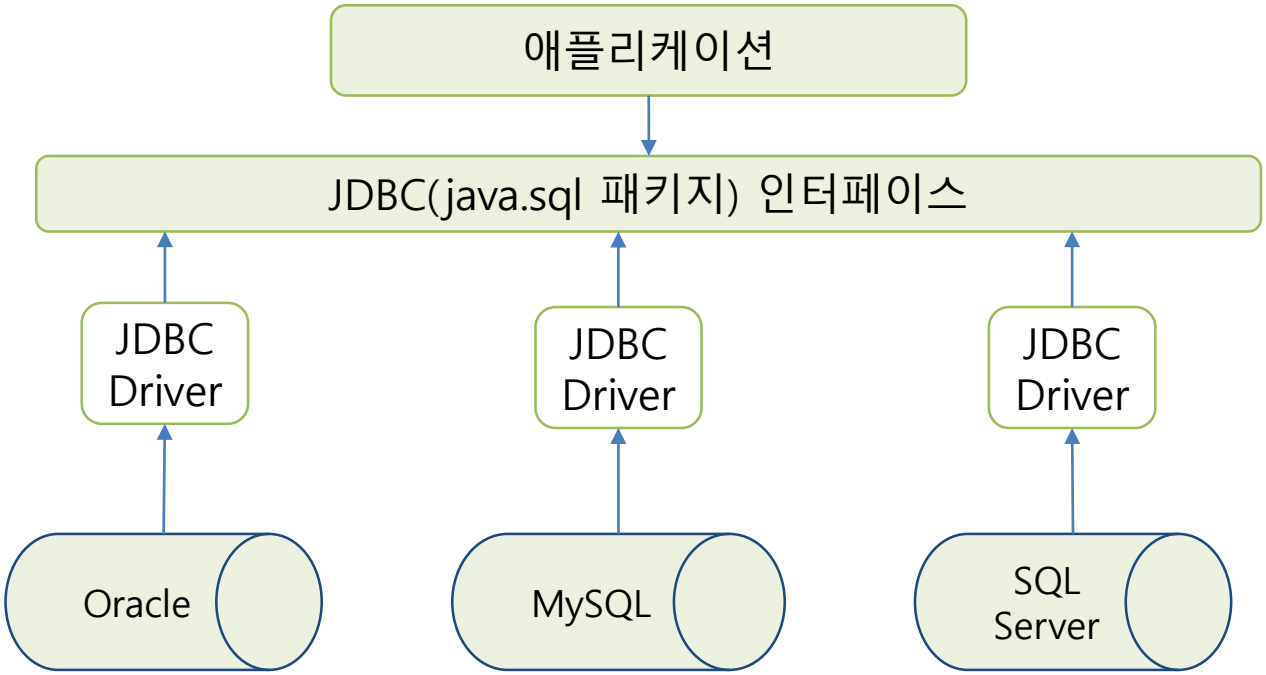
22강. 데이터베이스 입출력

- JDBC 개요
- DB 연결
- 데이터처리
- 프로시저와 함수 호출
- 트랜잭션처리

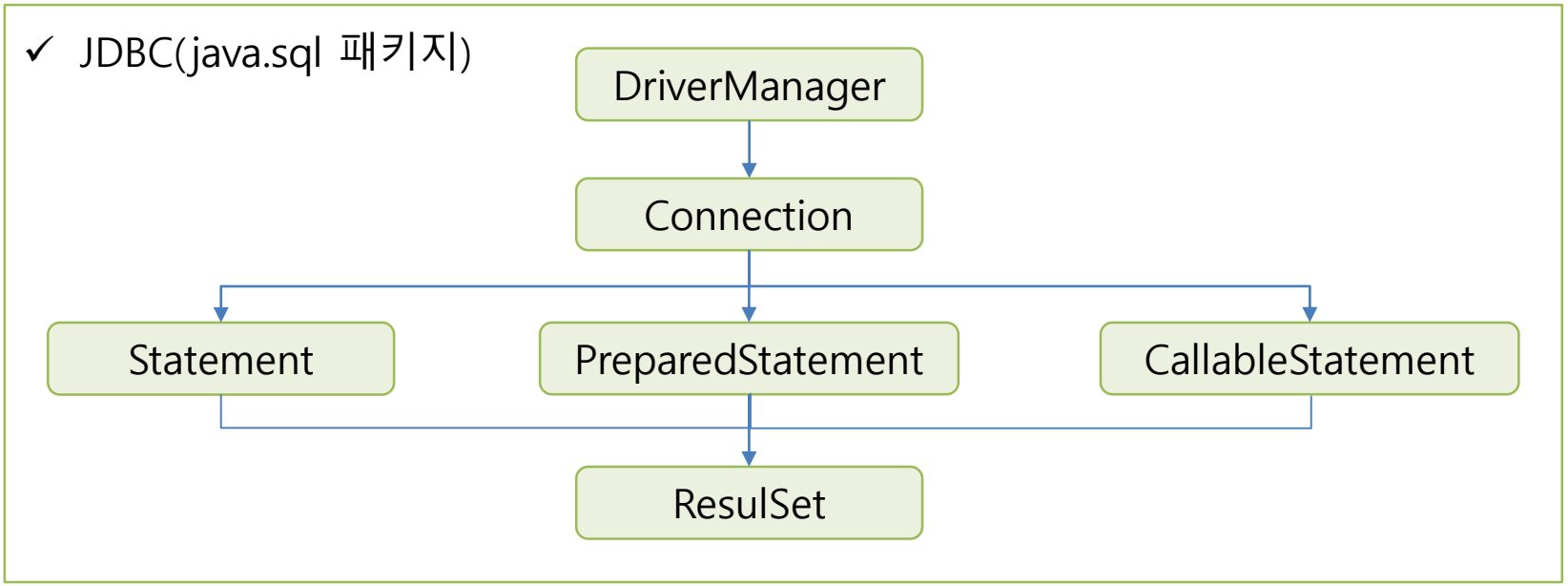
22-1. JDBC 개요

- 자바는 데이터베이스(DB)와 연결하여 데이터 입출력 작업을 할 수 있도록 JDBC(Java Database Connectivity) 라이브러리(java.sql 패키지)를 제공한다.
- JDBC는 데이터베이스 관리시스템(DBMS)의 종류와 상관없이 동일하게 사용할 수 있는 클래스와 인터페이스로 구성되어 있다.



- JDBC 인터페이스를 통해 실제로 DB와 작업하는 것은 JDBC Driver이다.
- JDBC Driver는 JDBC 인터페이스 구현체이다.

22-1. JDBC 개요



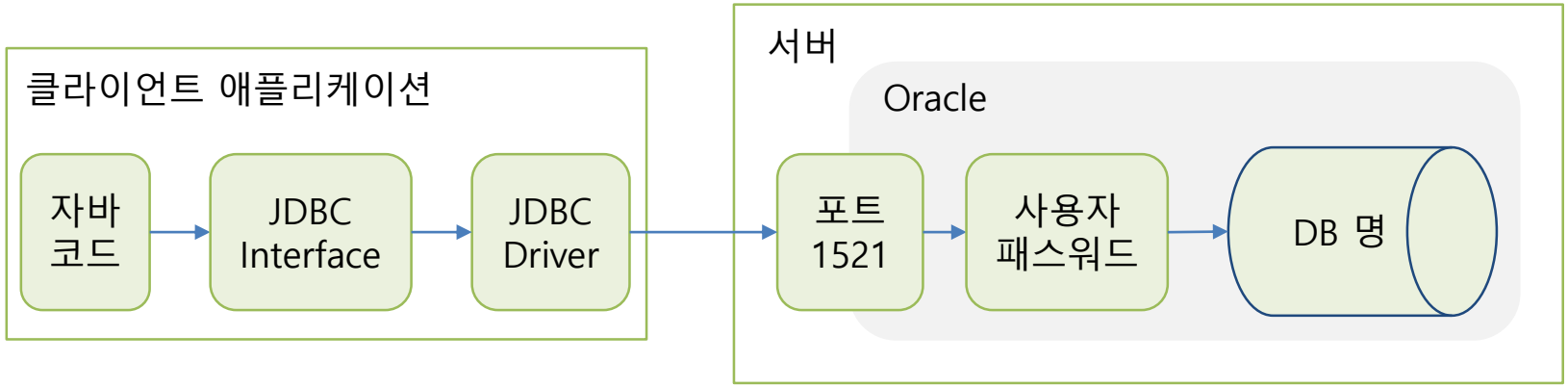
- DriverManager : 클래스로 JDBC Driver를 관리하며 DB와 연결, Connection 구현체 생성한다.
- Connection : 인터페이스로 Statement, PreparedStatement, CallableStatement 구현 객체를 생성하며, 트랜잭션 처리와 DB Connection을 끊을 때 사용한다.
- Statement : 인터페이스로 SQL의 DDL(Data Definition Language)과 DML(Data Manipulation Language) 실행한다. 정적 SQL문 실행 시 사용한다.

22-1. JDBC 개요

- PreparedStatement : 인터페이스로 SQL의 DDL(Data Definition Language)과 DML(Data Manipulation Language) 실행할 때 사용한다. 매개변수화된 SQL 문을 사용할 수 있어 편리성과 보안성이 좋다. SQL 작업 시 주로 사용한다.
- CallableStatement : DB에 저장되어 있는 프로시저(Procedure)와 함수(Function)를 호출할 때 사용한다.
- ResultSet : DB에서 가져온 데이터를 읽을 때 사용한다.

22-2. DB 연결

➢ Oracle DB 연결 구성도



- **JDBC Driver 설치**
  - 다운로드 URL : https://mvnrepository.com/artifact/com.oracle.database.jdbc/ojdbc8
  - 적절한 버전의 Oracle JDBC Driver를 다운 받아 프로젝트의 lib 폴더에 ojdbc8.jar 파일을 저장한다.

22-2. DB 연결

- **DB 연결**

1. JDBC Driver 메모리 로딩

```
Class.forName("oracle.jdbc.OracleDriver"); // 클래스 로더 사용
```

```
DriverManager.registerDriver(new oracle.jdbc.driver.OracleDriver());
 // Java9 이후 DriverManager 통한 JDBC 드라이브 로딩 지원
```

- JDBC Driver 를 찾지 못하면 ClassNotFoundException 이 발생(예외처리 해야 함)

2. DriverManager의 getConnection() 메서드로 DB 와 연결

```
Connection conn = DriverManager.getConnection("연결 문자열","사용자","비밀번호");
```

- Oracle 연결 문자열

```
jdbc:oracle:thin:@localhost:1521/orcl
```

TCP용 JDBC Driver 사용 | IP 주소 | 포트 | DB명

- 연결 성공 : getConnection() 메서드는 Connection 객체를 리턴
- 연결 실패 : SQLException 예외처리

3. DB 연결을 끊을 때는 Connection 객체의 close() 메서드 호출

22-2. DB 연결

➢ DB 연결 및 연결 끊기 사용 예

```
Connection conn = null;

try {
    Class.forName("oracle.jdbc.OracleDriver");

    String dbUrl = "jdbc:oracle:thin:@locahost:1521/orcl";
    String dbUser = "oracletest";
    String dbPaswd = "oraclepaswd";

    conn = DriverManager.getConnection(dbUrl, dbUser, dbPaswd);
    System.out.println("연결 성공");
} catch(ClassNotFoundException e) {
    System.out.println("Oracle JDBC 드라이버를 찾을 수 없습니다.");
    e.printStackTrace();
} catch(SQLException e) {
    e.printStackTrace();
} finally {
    if(conn != null) {
        try {
            // 연결 끊기
            conn.close();
        } catch(SQLException e) {
            e.printStackTrace();
        }
    }
}
```

22-3. 데이터 처리

- Statement 인터페이스

```
Statement stmt = conn.statement(sql);
```

  - stmt.executeQuery() : 데이터를 조회(SELECT)하는 경우 사용, 가져온 데이터를 ResultSet에 저장 후 리턴

```
ResultSet rs = stmt.executeQuery();
```

  - stmt.executeUpdate() : INSERT, UPDATE, DELETE SQL 문에 사용

```
int rows = stmt.executeUpdate();
```

  - stmt.close() : 사용이 끝난 경우 호출해서 사용했던 메모리를 해제한다.

- 정적 SQL문으로 사용자 입력한 데이터가 포함된 쿼리를 실행할 때는 적절한 이스케이프 처리가 필요하다.
- SQL 삽입 공격(SQL Injection) 등의 보안 문제가 발생할 수 있다.

22-3. 데이터 처리

- PreparedStatement 인터페이스

```
PreparedStatement pstmt = conn.preparedStatement(sql);
```

  - pstmt.executeQuery() : 데이터를 조회(SELECT)하는 경우 사용, 가져온 데이터를 ResultSet에 저장 후 리턴

```
ResultSet rs = pstmt.executeQuery();
```

  - pstmt.executeUpdate() : INSERT, UPDATE, DELETE SQL 문에 사용

```
int rows = pstmt.executeUpdate();
```

  - pstmt.close() : 사용이 끝난 경우 호출해서 사용했던 메모리를 해제한다.

- 매개변수화된 SQL문인 PreparedStatement는 미리 컴파일된(precompiled) SQL 문이다.
- 미리 컴파일된 쿼리를 사용하고 파라미터 바인딩('?' 와 setXXX() 메서드 사용)을 사용하여 편리성과 보안성이 좋다.

22-3. 데이터처리

- **데이터 저장(INSERT)**
  - USERS 테이블에 신규 사용자 정보 추가 SQL문

```
INSERT INTO USERS(ID_USER, NM_USER, ID_PASWD, NB_AGE, ID_EMAIL) VALUES('test1',
'홍길동', 'test1', 25, 'test1@test.com');
```

  - 값을 ?(물음표)로 대체한 매개변수화된 INSERT 문

```
INSERT INTO USERS(ID_USER, NM_USER, ID_PASWD, NB_AGE, ID_EMAIL)
VALUES(?, ?, ?, ?, ?);
```

  - INSERT 문을 String 타입 변수 sql에 문자열로 대입한다.

```
String sql = new StringBuilder()
            .append("INSERT INTO USERS(ID_USER, NM_USER, ID_PASWD, NB_AGE, ID_EMAIL)")
            .append("VALUES(?, ?, ?, ?, ?)")
            .toString();

또는

String sql = "INSERT INTO USERS(ID_USER, NM_USER, ID_PASWD, NB_AGE, ID_EMAIL)"
            + "VALUES(?, ?, ?, ?, ?)";
```

22-3. 데이터처리

- **데이터 저장(INSERT)**
  - PreparedStatement 명령문 생성 및 매개 값(?에 들어갈 값 지정) 지정

```
PreparedStatement pstmt = conn.preparedStatement(sql);
        pstmt.setString(1, "test1");
        pstmt.setString(2, "홍길동");
        pstmt.setString(3, "test1");
        pstmt.setInt(4, 25);
        pstmt.setString(5, "test1@test.com");
```

  - executeUpdate() 메서드 호출을 통한 SQL문 실행

```
int rows = pstmt.executeUpdate();
```

  - PreparedStatement 를 더 이상 사용하지 않을 경우 close() 메서드를 호출해서 메모리를 해제한다.

```
pstmt.close();
```

22-3. 데이터처리

- **데이터 저장(INSERT) – 사용 예**

```
String sql = "INSERT INTO USERS(ID_USER, NM_USER, ID_PASWD, NB_AGE, ID_EMAIL)"
        + "VALUES(?, ?, ?, ?, ?)";
PreparedStatement pstmt = conn.prepareStatement(sql);
pstmt.setString(1, "test1");
pstmt.setString(2, "홍길동");
pstmt.setString(3, "test1");
pstmt.setInt(4,  25);
pstmt.setString(5, "test1@test.com");

// SQL문 실행
int rows = pstmt.executeUpdate();
System.out.println("저장된 행 수: " + rows);

// PreparedStatement 닫기
pstmt.close();
```

22-3. 데이터처리

- **데이터 저장(INSERT) – Sequence 사용 및 조회, Blob 사용**

```
// 매개변수화된 SQL문 작성
String sql = "INSERT INTO BOARD(NB_BOARD, NM_TITLE, NM_CONTENT, NM_WRITER, DA_WRITE, NM_FILENAME, BO_FILEDATA)"
        + "VALUES(SEQ_BOARD.NEXTVAL, ?, ?, ?, SYSDATE, ?, ?)";

// PreparedStatement 열기 및 값 지정
PreparedStatement pstmt = conn.prepareStatement(sql, new String[] {"nb_board"});
pstmt.setString(1, "게시물 작성 테스트");
pstmt.setString(2, "게시물 내용");
pstmt.setString(3, "홍길동");
pstmt.setString(4, "sample1.png");
pstmt.setBlob(5, new FileInputStream("c:/Utility/SampleVideo/sample1.png"));

// SQL문 실행
int rows = pstmt.executeUpdate();
System.out.println("저장된 행 수: " + rows);

// nb_board 생성된 sequence 값 얻기
if(rows == 1) {
    ResultSet rs = pstmt.getGeneratedKeys();
    if(rs.next()) {
        int nbBoard = rs.getInt(1);
        System.out.println("nb_board(seq_board sequence 값: " + nbBoard);
    }
    // ResultSet close
    rs.close();
}

// PreparedStatement 닫기
pstmt.close();
```

22-3. 데이터처리

- **데이터 수정(UPDATE)**
  - UPDATE 문 SYNTAX

```
UPDATE table_name
SET column1 = value1, column2 = value2, ...
[WHERE condition]
```

  - UPDATE 문을 String 타입 변수 sql에 문자열로 대입한다.

```
String sql = new StringBuilder()
                .append("UPDATE board SET")
                .append("nm_title=?")
                .append("nm_content=?")
                .append("WHERE nb_board=?")
                .toString();
```

22-3. 데이터처리

- **데이터 수정(UPDATE) – 사용 예**

```
// 매개변수화된 SQL문 작성
String sql = new StringBuilder()
        .append("UPDATE board SET ")
        .append("nm_title=?, ")
        .append("nm_content=? ")
        .append("WHERE nb_board=? ")
        .toString();

// PreparedStatement 열기 및 값 지정
PreparedStatement pstmt = conn.prepareStatement(sql);
pstmt.setString(1, "게시물 수정 테스트");
pstmt.setString(2, "게시물 내용 수정");
pstmt.setInt(3, 5);

// SQL문 실행
int rows = pstmt.executeUpdate();    // 0: 수정된 행 없음
System.out.println("수정된 행 수: " + rows);

// PreparedStatement 닫기
pstmt.close();
```

22-3. 데이터처리

- **데이터 삭제(DELETE)**
  - DELETE 문 SYNTAX

```
DELETE FROM table_name
[WHERE condition]
```

  - DELETE 문을 String 타입 변수 sql에 문자열로 대입한다.

```
String sql = new StringBuilder()
             .append("DELETE FROM board ")
             .append("WHERE nb_board=?")
             .toString();
```

22-3. 데이터처리

- **데이터 삭제(DELETE) – 사용 예**

```
// 매개변수화된 SQL문 작성
String sql = new StringBuilder()
        .append("DELETE FROM board ")
        .append("WHERE nb_board=? ")
        .toString();

// PreparedStatement 열기 및 값 지정
PreparedStatement pstmt = conn.prepareStatement(sql);
pstmt.setInt(1, 3);

// SQL문 실행
int rows = pstmt.executeUpdate();    // 0: 삭제된 행 없음
System.out.println("삭제된 행 수: " + rows);

// PreparedStatement 닫기
pstmt.close();
```

22-3. 데이터처리

- **데이터 조회(SELECT)**
  - pstmt.executeQuery() : 데이터를 조회(SELECT)하는 경우 사용, 가져온 데이터를 ResultSet에 저장 후 리턴

```
ResultSet rs = pstmt.executeQuery();
```

  - ResultSet은 커서가 붙은 데이터들을 리턴하며, 첫번째 행부터 순차적으로 커서를 이동시켜 데이터를 읽어 나간다. 이 때 next() 메서드를 사용한다.

```
boolean result = rs.next();
```

  - ResultSet 사용이 끝나면 close() 메서드를 호출하여 닫는다.

```
rs.close();
```

22-3. 데이터처리

- **데이터 읽기(SELECT) – 사용 예**

```
// 매개변수화된 SQL문 작성
String sql = "SELECT nb_board, nm_title, nm_content, nm_writer "
        + "FROM board "
        + "WHERE nb_board IN(1, 4, 5)";

// PreparedStatement 열기 및 값 지정
PreparedStatement pstmt = conn.prepareStatement(sql);

// SQL문 실행
ResultSet rs = pstmt.executeQuery();
while(rs.next()) {
    System.out.println("-----------------------------------------------------");
    System.out.println("번호: " + rs.getInt(1));
    System.out.println("제목: " + rs.getString(2));
    System.out.println("내용: " + rs.getString(3));
    System.out.println("작성자: " + rs.getString(4));
    System.out.println("-----------------------------------------------------");
    System.out.println();
}
rs.close();          // ResultSet 닫기

// PreparedStatement 닫기
pstmt.close();
```

22-4. 프로시저와 함수 호출

- **프로시저**
  - Procedure 호출 SYNTAX

```
String sql = "{ call 프로시저명(?, ?, ...) }";
CallablesStatement cstmt = conn.prepareCall(sql);
```

  - IN: 2개, OUT: 1개를 가진 프로시저 ?(매개 값) 설정

```
String sql = "{ call 프로시저명(?, ?, ?) }";
CallablesStatement cstmt = conn.prepareCall(sql);
cstmt.setString(1, "값");     // 프로시저 첫번째 매개 값
cstmt.setString(2, "값");     // 프로시저 두번째 매개 값
cstmt.registerOutParameter(3, 리턴타입); // 세번째 는 OUT값(리턴값) 지정
```

  - 프로시저 호출

```
cstmt.execute();
```

  - close() 메서드로 닫기 및 메모리 해제 처리

```
cstmt.close();
```

22-4. 프로시저와 함수 호출

- **함수**
  - Function 호출 SYNTAX

```
String sql = "{ call 함수명(?, ?, ...) }";
CallablesStatement cstmt = conn.prepareCall(sql);
```

  - 리턴 값과 매개 값: 2개를 가진 함수 ?(리턴 값 및 매개 값) 설정

```
String sql = "{ ? = call user_login(?, ?) }";
CallablesStatement cstmt = conn.prepareCall(sql);
cstmt.registerOutParameter(1, Types.INTEGER); // 리턴값: PLS_INTEGER
cstmt.setString(2, "test");                   // 프로시저 첫번째 매개 값
cstmt.setString(3, "12345");                  // 프로시저 두번째 매개 값
```

  - 함수 호출

```
cstmt.execute();
int result = cstmt.getInt(1);  // 첫번째 ? 값 얻기, 함수의 리턴 값
```

  - close() 메서드로 닫기 및 메모리 해제 처리

```
cstmt.close();
```

22-5. 트랜잭션 처리

- 트랜잭션(Transaction)은 기능 처리의 최소 단위이다. 하나의 기능은 여러 개의 작은 작업들로 구성된다.
- 기능 처리의 최소 단위는 이 작은 작업들을 분리할 수 없고 전체를 하나라는 전제이다. 트랜잭션은 작은 작업들이 모두 성공하거나 실패해야 한다.
- DB에서 트랜잭션을 처리는 커밋(commit)과 롤백(rollback)이 있다.
  - 커밋(commit) : 트랜잭션에 묶인 여러 개의 작은 작업들을 모두 성공 처리
  - 롤백(rollback) : 트랜잭션에 묶인 여러 개의 작은 작업들을 모두 실패 처리

- JDBC는 DML(INSERT, UPDATE, DELETE) 실행 시 자동 커밋 처리된다. 트랜잭션을 사용하려면 자동 커밋을 꺼야 한다.

```
conn.setAutoCommit(false);
```

- 커밋과 롤백 제어

```
conn.commit();    // 커밋
conn.rollback();  // 롤백
```

22-5. 트랜잭션 처리

➢ 트랜잭션(Transaction)을 위한 코드 작성 패턴

```
Connection conn = null;

try {
    // Transaction 시작
    conn.setAutoCommit(false);              // 자동커밋 끄기

    // 작은작업처리
    ...
    // 작은작업처리
    ...

    // Conmmit 처리
    conn.commit();
    // Transaction 종료
} catch(Exception e) {
    try {
        // Rollback처리
        conn.rollback();
    } catch(SQLException e1) {}

} finally {
    try {
        if(conn != null) {
            conn.setAutoCommit(true);       // 자동커밋 켜기(원상태복구)
            conn.close();                   // 연결 끊기
        }
    } catch(SQLException e) {}
}
```

➢ 예제 1

JDBC에 대한 설명으로 틀린 것은 무엇입니까?

① java.sql에서 제공하는 표준 라이브러리를 말한다.
② DBMS의 종류와 상관없이 사용할 수 있는 클래스와 인터페이스로 구성되어 있다.
③ JDBC 인터페이스들을 구현한 것이 JDBC Driver이다.
④ JDBC Driver는 DBMS 종류와 상관없이 동일한 것을 사용할 수 있다.

➢ 예제 2

JDBC로 SQL실행 결과를 얻기 위한 코드 작성 순서는? ( )->( )->( )->( )

① DriverManager로부터 Connection을 얻는다.
② Class.forName() 메서드를 이용해서 JDBC Driver 클래스를 로딩한다.
③ ResultSet에서 SQL 실행 결과를 얻는다.
④ PreparedStatement를 얻고 SQL문을 실행한다.

➢ 예제 3

트랙잭션에 대한 설명으로 틀린 것은 무엇입니까?

① 기능 처리의 최소 단위를 말한다.
② 커밋(commit)은 내부 작업을 모두 성공 처리한다.
③ 롤백(rollback)은 내부 작업 중에서 성공한 작업까지 되돌린다.
④ 트랜잭션을 코드로 제어하려면 setAutoCommit(false) 메서드를 먼저 호출해야 한다.

감사합니다.